

ილიას სახელმწიფო უნივერსიტეტი
ბიზნესის, ტექნოლოგიისა და განათლების ფაკულტეტი
ტექნოლოგიების სკოლა

„ჭკვიანი თავშესაფარი“
პროექტის ანგარიში

გუნდის წევრები:

თამარ ჯაში
გრიშა ტონერიანი
მარი დავლაძე

კომპიუტერული ინჟინერიის საბაკალავრო პროგრამა
დამამთავრებელი კურსის პროექტი B (CE490B)

პროექტის ხელმძღვანელები: პროფ. დავით ჩხაიძე, პროფ. გიორგი მოდებაძე

6.18.2026

თბილისი, საქართველო

სარჩევი

აბსტრაქტი.....	3
თავი 1. შესავალი.....	4
თავი 2. ფონური მასალა	6
თავი 3. დიზაინის შემუშავება	10
თავი 4. საბოლოო დიზაინის აღწერა	15
თავი 5. დიზაინის ვერიფიკაციის გეგმა (ტესტირება)	23
თავი 6. დასკვნები და რეკომენდაციები	25
გამოყენებული წყაროები.....	27
დანართები.....	28

ნახაზების ჩამონათვალი

ნახაზი 1. 3D მოდელი	15
ნახაზი 2. სტრუქტურული და ფუნქციური ბლოკ-სქემა	16
ნახაზი 3. Wi-Fi კავშირის დამყარების ბლოკ-სქემა.....	20
ნახაზი 4. თერმოსტატის ალგორითმის ბლოკ-სქემა	21
ნახაზი 5. მონაცემების გადაცემის ბლოკ-სქემა	21
ნახაზი 6. ელექტრო-სქემა	22
ნახაზი 7. ელექტრული წრედი	23
ნახაზი 8. მონტაჟის სქემა	23
ნახაზი 9. საწარმოო ნახაზები.....	24

ცხრილების ჩამონათვალი

ცხრილი 1. VL53L1X ToF სენსორის ტექნიკური მახასიათებლები.....	06
ცხრილი 2. SHT40 სენსორის ტექნიკური მახასიათებლები.....	07
ცხრილი 3. მიკროკონტროლერების შედარება.....	08
ცხრილი 4. გამოსაყენებელი სტანდარტების ჩამონათვალი.....	08
ცხრილი 5. კომპონენტების შერჩევის მატრიცა.....	10
ცხრილი 6. ენერგობიუჯეტი.....	12
ცხრილი 7. საბოლოო მასალების ნუსხა (BOM).....	19
ცხრილი 8. სპეციფიკაციების შემოწმების სია.....	27
ცხრილი 9. განტის დიაგრამა.....	28
ცხრილი 10. ინდივიდუალური წვლილი.....	29
ცხრილი 11. კომპონენტების სპეციფიკაციები და datasheet-ები.....	31

აბსტრაქტი / მოკლე მიმოხილვა

„ჭკვიანი თავშესაფარი“ ავტომატიზებული, IoT სისტემაა, რომელიც შეიქმნა ისეთი სოციალური პრობლემის გადასაჭრელად, როგორიცაა ზამთარში მიუსაფარი ცხოველების კლიმატური პირობების გამო სიკვდილიანობის გაზრდა. სისტემა სენსორების მონაცემებზე დაყრდნობით ავტომატურად რთავს და თიშავს გათბობას. ასევე პროექტი მოიცავს თავშესაფრის დისტანციურ მონიტორინგს და მოხალისეებს აძლევს დონაციის გაკეთების საშუალებას.

ძირითადი მიკროკონტროლერია ESP32-S3, რომელიც LilyGo T-SIMCAM პლატაზეა განთავსებული, OV5640 კამერასთან ერთად. ეს კომპონენტი პასუხისმგებელია სენსორული ინფორმაციის წაკითხვაზე, გადაწყვეტილების მიღებაზე, რელეს მართვაზე სერვერთან სტაბილური კავშირის დამყარებასა და ვიდეო სტრიმის და მონაცემების სერვერზე ატვირთვაზე. ESP32-S3 მიკროკონტროლერს სენსორულ მონაცემებს UART პროტოკოლით უგზავნის STM32F103 BluePill, რომელიც I2C და UART პროტოკოლებით იღებს მანძილის და ტემპერატურის მონაცემებს VL53L1X ToF და SHT40 სენსორებისგან. OV5640 კამერის მიერ გადაღებული ვიდეონაკადი ეგზავნება ESP32-ს, რომელიც მას RTSP ნაკადად გარდაქმნის და ისე უგზავნის Digital Ocean სერვერზე არსებულ MediaMTX სერვერს. Go ენაზე დაწერილი ბექენდი Digital Ocean-ის VM-ზე მუშაობს, TCP პორტზე იღებს სენსორულ მონაცემებს და ასახავს ვებსაიტზე. გათბობა ირთვება მხოლოდ მაშინ, როდესაც ტემპერატურა 10°C -ზე დაბლა ეცემა და ცხოველი თავშესაფარშია.

თავშესაფრის კონსტრუქცია აგებულია ხის ფირფიტებისგან და სახურავის გადასაფარად გამოყენებულია ტოლი. გავაკეთეთ ორქანობიანი სახურავი, რათა არ მოხდეს ნალექის დაგროვება.

პროტოტიპმა სპეციფიკაციები წარმატებით დააკმაყოფილა. ტემპერატურის გაზომვის სიზუსტემ შეადგინა $\pm 0.3^{\circ}\text{C}$ (მოთხოვნა: $\pm 1^{\circ}\text{C}$), გათბობის რეაქციის დრომ 1.5 წამი (მოთხოვნა: ≤ 1.5 წამი), სენსორული მონაცემები კი სერვერზე გადაიცემა ყოველ წამში. 30W-იანი სიმძლავრის გამათბობელი ელემენტები საკმარისია $50 \times 50 \times 50$ სმ კონსტრუქციის გასათბობად. პროექტი ადასტურებს, რომ ბიუჯეტური IoT სისტემა ეფექტურად ჭრის მიუსაფარი ცხოველების უსაფრთხოების პრობლემას.

თავი 1. შესავალი

1.1 ფონი და საჭიროებები

საქართველოში ბოლო წლების განმავლობაში მნიშვნელოვნად გაიზარდა მიუსაფარი ცხოველების რიცხვი, შესაბამისად უფრო მეტი მათგანი ხდება ჰიპოთერმიის მსხვერპლი ზამთარში. სახელმწიფო საჭირო პირობებს ვერ ქმნის მიუსაფარი ცხოველების ყინვისგან დასაცავად. მოხალისეები და ცხოველთა დაცვის ორგანიზაციები ცდილობენ შეუმსუბუქონ ქუჩის ბინადრებს ცხოვრება ხის ან მუყაოს კონსტრუქციების ქალაქსა თუ რეგიონებში განლაგებით, თუმცა ეს ხშირად არ არის საკმარისი მათი უსაფრთხოების უზრუნველსაყოფად. თავშესაფარში საჭიროა ჩაშენებული იყოს აქტიური, ავტომატური გათბობის მოდული, რომელიც მოერგება ცხოველის საჭიროებებს და არა პირიქით. მიუსაფარი ცხოველების გათბობით და თავშესაფრით უზრუნველყოფა შეამცირებს მათ

სიკვდილიანობას. გადაწყვეტიდან სარგებელს მიიღებენ როგორც უშუალოდ ცხოველები, ისე მოხალისე ორგანიზაციები, რომლებსაც ავტომატური გათბობის სისტემა სამუალებას მისცემთ რესურსები დაზოგონ.

1.2 პრობლემის ფორმალური განსაზღვრება

საჭირო იყო შეგვექმნა სისტემა, რომელიც ავტომატურად გაათბობს თავშესაფარს, მხოლოდ მაშინ როცა ორი პირობა სრულდება - ცხოველი შიგნითაა და ტემპერატურა ზღვარს ქვემოთ ეცემა. ამასთანავე, სისტემამ უნდა უზრუნველყოს ვიდეო მონიტორინგი, რომ მოსახლეობას აუმაღლდეს ცნობიერება ამ საკითხზე და მიეცეს შესაძლებლობა თვალი ადევნოს ცხოველებს ლაივში, სურვილის შემთხვევაში კი გააკეთოს დონაცია მათ დასახარებლად. სისტემის საზღვრებში შედის მანძილის და ტემპერატურის სენსორები, გამათბობელი ფირი, მონაცემების გადაცემა ვებსერვერზე, ვიდეოსტრიმი და ვებ - პლატფორმა მონიტორინგისა და შემოწირულობისთვის. სისტემა არ მოიცავს ცხოველის ამოცნობას, მისი ჯანმრთელობის მონიტორინგს ან გამოკვებას.

კომპონენტების ჯამური ღირებულება შეადგენს 378.09 ლარს. სისტემაში გამოყენებული ელექტრონიკა (მიკროკონტროლერი, კამერა, სენსორები) 5V - ზე მუშაობს, გათბობის ფირი კი 12V-ზე. ელექტრონიკას და გათბობის მოდულს გააჩნიათ ორი სხვადასხვა კვების წყარო. ელექტრონიკა განთავსებულია თავშესაფრის შიგნით, წვიმის, თოვლის და სხვა დამაზიანებელი გარემოებებისგან დაცულ სივრცეში. სისტემა და გამათბობელი მოდული მოთავსებულია 50x50x50 სმ ზომის ხის თავშესაფარში.

1.3 მიზნებისა და სპეციფიკაციების შემუშავება

1. ტემპერატურა - გათბობა ირთვება $<10^{\circ}\text{C}$ -ზე, ითიშება $>12^{\circ}\text{C}$ -ზე
2. SHT40 სენსორის სიზუსტე - ტემპერატურის გაზომვის სიზუსტე არის $\pm 1^{\circ}\text{C}$ ის ფარგლებში
3. რეაგირების დრო - გათბობის ჩართვა ან გამორთვა ხდება ≤ 1.5 წამში პირობის დაფიქსირებიდან
4. ვიდეოსტრიმის დაყოვნება - ვიდეოსტრიმი ბრაუზერში ჩნდება ≤ 3 წმ -ის დაყოვნებით
5. ცხოველის დაფიქსირება - 500მმ-ზე ნაკლები თუა მანძილი სენსორსა და უახლოეს ობიექტს შორის, სისტემა ჩათვლის რომ თავშესაფარი დაკავებულია
6. სისტემის სიმძლავრე - სისტემის საერთო სიმძლავრე არ აღემატება 31W-ს (გათბობის ელემენტები 30W + ელექტრონიკა 1W)
7. სერვერზე მონაცემების გაგზავნა - ESP32 JSON პაკეტს სერვერზე აგზავნის ყოველ ერთ წამში

თავი 2. ფონური მასალა

2.1 არსებული პროდუქტები

ბაზარზე არსებობს მსგავსი პროდუქტები, მაგ. თავშესაფრები ავტომატიზებული სისტემით: K&H Pet Products[1], Petkit House[2] და სხვა. PetKit - ის სახლები შიდა პირობებისთვისაა განკუთვნილი და ვერ გაუძლებს ქუჩის პირობებს, განსაკუთრებით ზამთარში, თოვლის და წვიმის დროს. K&H Pet Products - გარე მოხმარებისაა, თუმცა არ გააჩნია კამერა მონიტორინგისთვის და არ აქვს ინტერნეტთან კავშირი. ჩვენი პროექტი აერთიანებს ავტომატურ გათბობას, გარე პირობებისადმი მდგრადობას(გამოვიყენეთ ხე კედლებისთვის და ტოლი სახურავისთვის) და გააჩნია პლატფორმა, რომელიც მოხალისეებს საშუალებას აძლევს თვალი ადევნოს და დაეხმაროს ცხოველებს.

2.2 კონკრეტული ტექნიკური მონაცემები

Time-of-Flight (ToF) გაზომვის პრინციპი

სენსორი აგზავნის ინფრაწითელ ლაზერულ იმპულსებს და ზომავს დროს იმ მონაკვეთს, რომელიც მის მიერ გაგზავნილ იმპულსს სჭირდება უახლოეს ობიექტამდე (მაგ.ცხოველამდე) მისასვლელად და უკან დასაბრუნებლად. [3]

მანძილი გამოითვლება შემდეგი ფორმულით: $d = (c \cdot t) / 2$, სადაც d მანძილია, ხოლო c სინათლის სიჩქარის მუდმივა.

ცხრილი N1

პარამეტრი	მნიშვნელობა
სამუშაო ძაბვა	3.3V - 5V
დენის მოხმარება	19 mA
მაქსიმალური მანძილი	4 მ
სამუშაო ტემპერატურა	-20°C - +70°C
I2C მისამართი	0x29

ტემპერატურის გაზომვის პრინციპი

SHT40 სენსორი ზომავს ტემპერატურას. პროექტში გამოყენებულია UART Bridge მოდული, რომლებიც მონაცემებს 9600 -baud სიჩქარით გადასცემს მიკროკონტროლერს ფორმატში: R:041.6RH 024.7C

სენსორი ტემპერატურას ტევადობის მეთოდით ზომავს. სენსორის შიგნით არის პატარა ელემენტი, რომლის ტევადობა იცვლება ტემპერატურის მიხედვით. სენსორი ამ ცვლილებას ზომავს და გარდაქმნის რიცხვად. [4]

ცხრილი N2

პარამეტრი	მნიშვნელობა
სამუშაო ძაბვა	1.08V – 3.6V
ტემპერატურის სიზუსტე	$\pm 0.2^{\circ}\text{C}$
სამუშაო ტემპერატურა	$-40^{\circ}\text{C} - +125^{\circ}\text{C}$

თერმოსტატის ალგორითმი

გათბობა ირთვება: ტემპერატურა $< 10^{\circ}\text{C}$ და ცხოველი შიგნით არის შესული

გათბობა ითიშება: ტემპერატურა $> 12^{\circ}\text{C}$ ან ცხოველი არ შესულა შიგნით

მდგომარეობის ცვლა ხდება 3 თანმიმდევრული დადასტურების შემთხვევაში

ვიდეოსტრიმინგის პროტოკოლები

OV5640 კამერა აგენერირებს JPEG კადრებს. ESP32 LilyGo კადრებს RTSP ფორმატში აგზავნის^[5] MediaMTX სერვერზე. Ffmpeg ნაკადს გადაიყვანს H.264 ვიდეო ფორმატში, ხოლო MediaMTX მიღებულ H.264 ნაკადს WebRTC პროტოკოლის სახით აწვდის ბრაუზერს.

UART პროტოკოლი

UART არის ასინქრონული სერიული კომუნიკაციის პროტოკოლი, ანუ მონაცემების გადაცემა ხდება საერთო ტაქტური სიგნალის გარეშე. ორივე მხარე წინასწარ შეთანხმებულ baud rate - ზე მუშაობს. მოცემულ პროექტში მონაცემთა გადაცემა ხორციელდება 9600 baud სიჩქარით BluePill (PA2) - ESP32 (GPIO46) ხაზზე, 1Hz სიხშირით (ანუ, წამში ერთხელ).

I2C პროტოკოლი

I2C არის სინქრონული, ორსადენიანი საკომუნიკაციო პროტოკოლი, რომელიც იყენებს მონაცემების - SDA და ტაქტურ - SCL ხაზებს. ჩვენს მიერ აგებულ სისტემაში BluePill მიკროკონტროლერი, რომელიც სენსორების მიერ გადაცემული ინფორმაციის წასაკითხად და ESP32 LilyGo- სთვის გადასაცემად გამოიყენება, არის Master - ი, ხოლო ToF სენსორი არის Slave -ი. რადგან ამ პროტოკოლის საკომუნიკაციო ხაზები open drain არქიტექტურისაა, სიგნალის სტაბილიზაციისთვის მათზე დამატებული გავქვს pull up რეზისტორები. სენსორიდან მონაცემების წაკითხვა ხდება სტანდარტულ - 10kHz სიჩქარეზე.

მიკრომარშრუტიზატორები [6, 7]

ცხრილი N3

პარამეტრი	STM32F103 (BluePill)	ESP32-S3
არქიტექტურა	ARM Cortex-M3	Xtensa LX7 (dual-core)
სიხშირე	72 MHz	240 MHz
Flash	64 KB	8 MB
RAM	20 KB SRAM	8MB PSRAM
სამუშაო ძაბვა	3.3 V	3.3V
კომუნიკაცია	UART, I2C, SPI	UART, WI-FI, SPI
OS	Bare-metal HAL	FreeRTOS

2.3 გამოსაყენებელი სტანდარტების ჩამონათვალი

ცხრილი N4

IEEE 802.11 (Wi-Fi)	განსაზღვრავს ESP32-S3 კავშირის პროტოკოლს STA რეჟიმში. პროექტში კონფიგურირებულია 3 SSID. თუ ერთი ქსელი მიუწვდომელია, სისტემა ავტომატურად სხვაზე გადაერთვება.
RFC 2326 (RTSP)	ESP32 MediaMTX სერვერთან კავშირს ამყარებს RTSP handshake - ით. 1. OPTIONS 2. ANNOUNCE 3. SETUP 4. RECORD
RFC 2435 (RTP/JPEG)	კამერის JPEG კადრები იფუთება RTP პაკეტებში და TCP -ით ეგზავნება სერვერს.
HTTP/1.1 (RFC 7230)	Go backend -ი და ESP32 - ის ლოკალური სერვერი HTTP-ზე მუშაობს. /api/sensor, /relay, /stream, ყველა ენდფოინტი HTTP GET/POST მეთოდებს იყენებს.

JSON (ECMA-404)	ESP32 სენსორების მონაცემებს JSON-ის სახით გზავნის Go სერვერზე. ფორმატი განსაზღვრავს მონაცემთა სტრუქტურას.
ARM CMSIS	STM32F103-ის HAL ბიბლიოთეკა CMSIS-ზეა დაფუძნებული. UART და I2C დრაივერები CMSIS-ის სტანდარტული ინტერფეისებით წერია.
IEC 62368-1	5V და 12V კონტურები ოპტიზოლაციით გამოყოფილია, რომ დაბალი ძაბვის სქემა სიმძლავრის კონტურს არ შეეხოს.

თავი 3. დიზაინის შემუშავება

3.1 კონცეპტუალური დიზაინების განხილვა

კონცეფცია N1 იდეა გვგეჟონდა გამოგვეყენებინა TapoC310 კამერა. ეს არის IP კამერა, RTSP მხარდაჭერით. მისი უპირატესობაა მაღალი გარჩევადობა და მცირედი დაყოვნება, ხოლო ნაკლოვანებაა ცალკე კვების წყაროს საჭიროება და ESP32-თან ინტეგრაციის სირთულე.

კონცეფცია N2 იდეა გვგეჟონდა გამოგვეყენებინა მხოლოდ STM32 მიკროკონტროლერი, ძირითადი ESP32 -S3 - ის ნაცვლად. მისი უპირატესობა იქნებოდა მარტივი არქიტექტურა და ნაკლები კომპონენტი. ნაკლოვანებაა ჩამენებული Wi-Fi და კამერის მოდულების არარსებობა.

კონცეფცია N3 იდეა გვგეჟონდა გამოგვეყენებინა SIM800C GSM მოდული, ტელემეტრიული მონაცემების სერვერისთვის გადასაცემად. მისი უპირატესობაა ის, Wi-Fi ქსელისგან დამოუკიდებელია და შეუძლია დაამყაროს სტაბილური კავშირი ინტერნეტის გარეშე. ნაკლოვანებები კი არის ის, რომ მოითხოვს SIM ბარათს და ყოველთვის სააბონენტო გადასახადს. STM32 - თან მისი ინტეგრაცია რთულია, საჭიროა AT ბრძანებების პარსინგი.

კონცეფცია N4 იდეა გვგეჟონდა გამოგვეყენებინა ერთი დიდი გამათბობელი ფირი. უპირატესობა იქნებოდა სითბოს იატაკზე თანაბარი გადანაწილება, ხოლო ნაკლოვანება სახლის ზომაზე მორგების სირთულე.

კონცეფცია N5 იდეა გვგეჟონდა გამოგვეყენებინა SSR რელე. მისი უპირატესობებია ხმაურის არარსებობა და ხანგრძლივად მუშაობის უნარი. ნაკლოვანებას წარმოადგენს მაღალი ფასი და დაბალი სიმძლავრის დატვირთვაზე დამატებით გაგრილების საჭიროება.

3.2 კონცეფციის შერჩევა

თითოეული კომპონენტისთვის შევადგინეთ შეფასების მატრიცა, სადაც შევადარეთ გამოყენებული მასალა და ალტერნატიული არჩევანი. შევაფასეთ ოთხი კრიტერიუმის

მიხედვით: ღირებულება, განხორციელებადობა, უსაფრთხოება და ინტეგრაცია.
შევაფასეთ 1-5 სკალაზე.

LilyGo T-SIMCAM შეირჩა, რადგან მასზე ჩაშენებული OV5640 კამერა პირდაპირ ESP32 პლატაზეა მოთავსებული, რაც ამცირებს კომპონენტების რაოდენობას და ფინანსურ დანახარჯს.

მიკროკონტროლერად LilyGo T-SIMCAM (ESP32-S3) შეირჩა, რადგან ერთ პლატაზე აერთიანებს Wi-Fi, კამერას და საკმარის გამოთვლით სიმძლავრეს.

კავშირისთვის Wi-Fi შევარჩიეთ SIM800C GSM - ის ნაცვლად, რადგან ღირებულება დაბალია, ინტეგრაცია მარტივია და SIM ბარათის ხარჯი არ ემატება.

გათბობისთვის პატარა ელემენტები(სტრინგები) შევარჩიეთ, რადგან უფრო ადვილად ერგება სახლის ზომას და ერთ-ერთის გაფუჭების შემთხვევაში ნაწილობრივ შეიძლება შეიცვალოს.

რელეს შემთხვევაში ოპტიზოლირებული ვარიანტი შეირჩა, რადგან იაფია და ამ დატვირთვისთვის საკმარისი.

კომპონენტების შერჩევის მატრიცა

ცხრილი N5

კამერა

კრიტერიუმი	LilyGo ჩაშ. კამერა	TapoC310
ღირებულება	5	3
განხორციელებადობა	5	5
უსაფრთხოება	4	4
ინტეგრაცია	5	2
ჯამი	19	14

მიკროკონტროლერი

კრიტერიუმი	LilyGo T-SIMCAM (ESP32 – S3)	STM32
ღირებულება	5	2

განხორციელებადობა	5	2
უსაფრთხოება	4	4
ინტეგრაცია	5	4
ჯამი	19	12

კავშირი

კრიტერიუმი	Wi-Fi (LilyGo)	SIM800C GSM
ღირებულება	5	2
განხორციელებადობა	5	3
უსაფრთხოება	4	3
ინტეგრაცია	5	3
ჯამი	19	16

რელე

კრიტერიუმი	ოპტიზირებული	SSR
ღირებულება	5	2
განხორციელებადობა	5	3
უსაფრთხოება	5	3
ინტეგრაცია	5	3
ჯამი	20	11

გამათბობელი ელემენტები

კრიტერიუმი	გამათბობელი ელემენტები	მთლიანი ფირი
ღირებულება	4	3
განხორციელებადობა	5	3
უსაფრთხოება	4	3
ინტეგრაცია	5	3
ჯამი	18	12

3.3 დამხმარე წინასწარი ანალიზი

ენერგობიუჯეტი

ცხრილი N6

კომპონენტი	ძაბვა	დენი	სიმძლავრე
ESP32-S3	3.3V	240mA(პიკური)	0.8W
STM32F103	3.3V	30mA	0.1W
VL53L1X	3.3V	19mA	0.06W
SHT40	3.3V	2mA	0.007W
ელექტრონიკა ჯამში	5V		1W
გამათბობელი ელემენტები	12V	2.5A	30W
სისტემა ჯამში (გათბობა ჩართული)			31W

ელექტრონიკა მოიხმარს მხოლოდ 1W-ს, გათბობა კი 30W-ს. შესაბამისად, სტანდარტული 5V/2A და 12V/5A კვების წყაროები საკმარისია. სისტემა ენერგეტიკულად განხორციელებადია.

დროითი ანალიზი - UART კომუნიკაცია

გადასაცემი შეტყობინება არის შემდეგი: S:T=23.45 H=41.23 D=1238 0=0 ST=1 SV=1/r/n

გადაცემის დრო: $42 \cdot 10 / 9600 = 44 \text{ ms}$

გადაცემა იკავებს 1000ms პერიოდის 4.4%-ს. შესაბამისად, 9600 baud საკმარისია და კავშირი არ ქმნის დაყოვნებას.

დროითი ანალიზი - თერმოსტატის რეაგირება

მინიმალური რეაგირების დრო: $3 \cdot 500 \text{ ms} = 1.5 \text{ წამი}$

ტემპერატურა რეალურ გარემოში წუთების განმავლობაში იცვლება, 1.5 წამიანი რეაგირება საკმარისია.

თერმული შეფასება - გათბობის საკმარისობა

სახლის ზომა არის 50x50x50 სმ. მასალა არის ხე. ($K=0.12 \text{ W/m}^2\text{K}$, სისქე 20მმ)

ზედაპირი: $6 \cdot (0.5 \cdot 0.5) = 1.5 \text{ m}^2$

$Q = K \cdot A \cdot \Delta T / d = 0.12 \cdot 1.5 \cdot 10 / 0.020 = 90 \text{ W}$

ცხოველის სხეული დამატებით 50-100W სითბოს გამოიმუშავებს. 30W(გათბობის სტრინგები) + 50-100W(ცხოველი) = 80-130W. ეს საკმარისია 90W დანაკარგის დასაფარად. 30W საკმარისია, რაგდან ცხოველი თავადაც ათბობს სივრცეს.

3.4 Proof of Concept ანალიზი ან ტესტირება

1. თავიდან კამერა შევამოწმეთ. LilyGo T-SIMCAM-ზე OV5640 მოდულის ინიციალიზაცია და MJPEG სტრიმინგი გავუშვით დამოუკიდებლად. ამ პროცესით დავრწმუნდით რომ პლატა და კამერა გამართულად მუშაობს.

2. შემდეგ ვცადეთ VL53L1X - ისა და SHT40-ის ძირითად მიკროკონტროლერზე - ESP32-S3-ზე მიერთება. ამ მცდელობამ გვაჩვენა, რომ ESP32-S3 - ის GPIO 26 -იდან GPIO 32- ის ჩათვლით პინები შიდა SPI Flash მეხსიერების მიერაა დაკავებული. შესაბამისად, მათ სენსორებისთვის ვერ გამოვიყენებდით და თავისუფალი პინების არარსებობის გამო, პირდაპირი კავშირი ვერ მოხდებოდა. სწორედ ამ მცდელობის შემდეგ გადავწყვიტეთ გამოგვეყენებინა STM32F103 BluePill სენსორული ინფორმაციის მისაღებად და ძირითად მიკროკონტროლერზე გადასაგზავნად.

3. თითოეული სენსორი ცალ-ცალკე დავტესტეთ BluePill-ზე, ჯერ VL53L1X I2C პროტოკოლით და შემდეგ SHT40 UART-ით.

4. სენსორების ინტეგრაციამდე შევამოწმეთ BluePill-სა და ESP32-ს შორის UART კავშირი. გავაგზავნეთ საცდელი შეტყობინებები და დავრწმუნდით რომ მონაცემები 9600 baud-ზე სწორად გადაიცემა.

5. ასევე დავტესტეთ Wi-Fi კავშირი Digital Ocean სერვერთან. ESP32 წარმატებით დაუკავშირდა სერვერს და დაიწყო მონაცემების TCP:5678 პორტზე გაგზავნა.

3.5 გამოსაყენებელი საინჟინრო სტანდარტების განხილვა

1. **ვიდეოსტრიმინგის არქიტექტურა (RFC 2326 და RFC 2435):** აღნიშნული სტანდარტები პირდაპირ გამოვიყენეთ rtsp_publisher.c მოდულის სტრუქტურის ასაგებად. კავშირის დამყარების თანმიმდევრითა და RTP პაკეტების ფორმატი ამ რეგულაციების მიხედვით დავაპროგრამეთ. ამის წყალობით, ჩვენი სისტემა თავსებადი გახდა MediaMTX სერვერისთვის. [\[5\]](#)

2. **მონაცემთა მიმოცვლის ფორმატი (HTTP/1.1 და JSON):** ეს სტანდარტები შევარჩიეთ ESP32-სა და ჩვენს Go სერვერს შორის კომუნიკაციისთვის, რამაც გაგვიმარტივა ბექენდის წერა და უზრუნველყო რომ ნებისმიერ ბრაუზერს შეეძლოს ამ მონაცემების წაკითხვა.

3. **დაბალი დონის პროგრამირება (ARM CMSIS):** ARM CMSIS სტანდარტზე დაყრდნობით ბევრად უფრო მარტივად დავწერეთ STM32F103 მიკროკონტროლერისთვის საჭირო UART და I2C დრაივერები. ერთნაირი ინტერფეისების წყალობით, კოდი გამოვიდა ადვილად წაკითხვადი და გამართვადი.

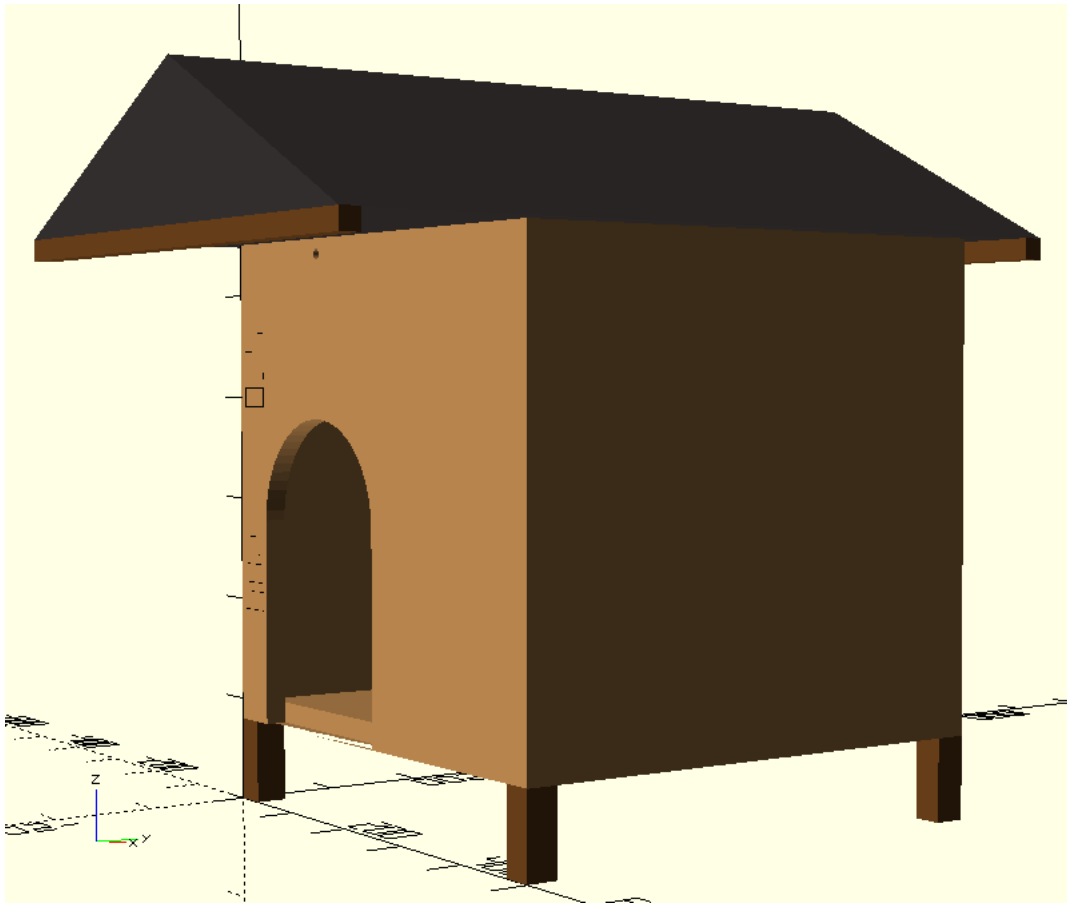
4. **აპარატურული უსაფრთხოება (IEC 62368-1):** ელექტროუსაფრთხოების გამო გადაწყვიტეთ პროექტში ოპტიმიზირებული რელე გამოგვეყენებინა, ჩვენთვის მნიშვნელოვანი იყო 5V იანი ციფრული კონტურისა და 12V-იანი ხაზის გალვანური გამიჯვნა, რომ დაგვეცვა თავშესაფრის უსაფრთხოება.

თავი 4. საბოლოო დიზაინის აღწერა

4.1 საერთო აღწერა / განლაგება

ჭკვიანი თავშესაფარი წარმოადგენს IoT სისტემას, რომელიც სენსორების მონაცემებზე დაყრდნობით მართავს გათბობას, ხოლო მონაცემებს და ლაივ ვიდეო სტრიმს კი გადასცემს სერვერს და ასახავს ვებსაიტზე. სისტემის მიზანია მიუსაფარ ცხოველებს სითბო მიაწოდოს ადამიანის ჩარევის გარეშე, მხოლოდ მაშინ როცა სჭირდებათ. ამით ენერგია იხარჯება მხოლოდ საჭიროებისამებრ.

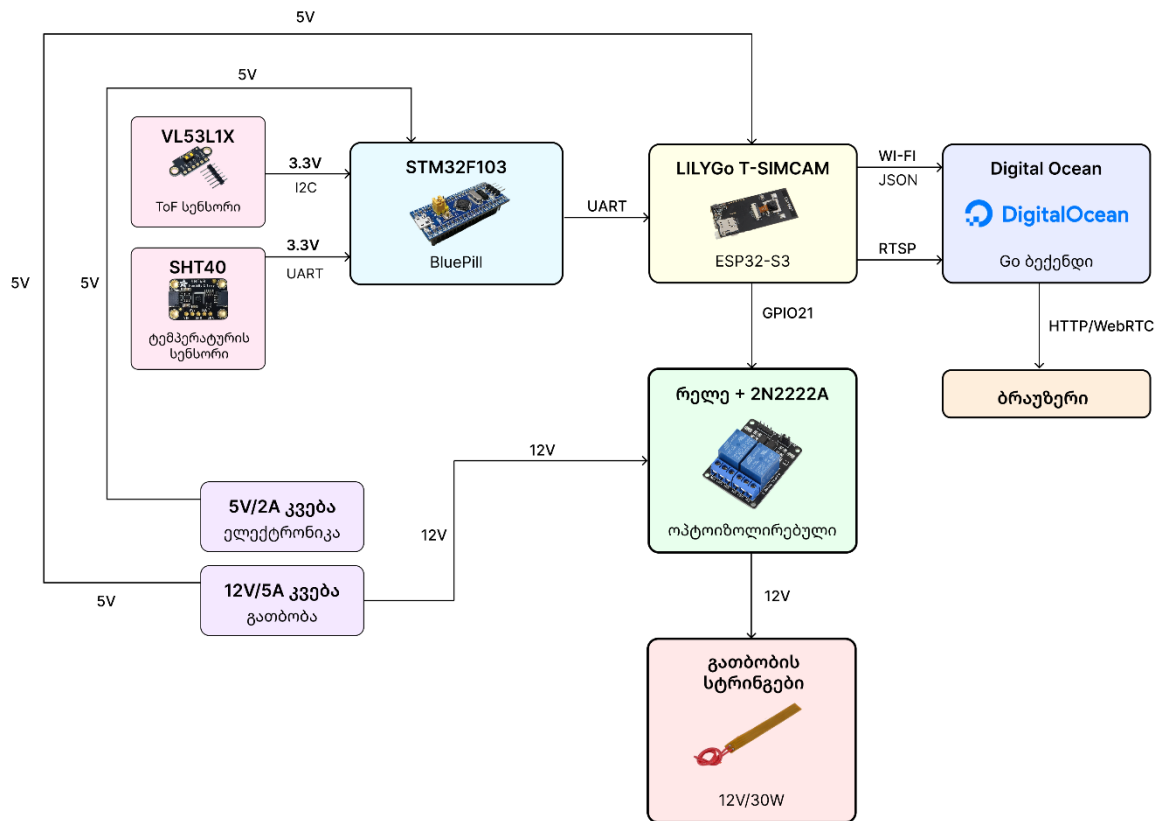
1. 3D მოდელი (ნახაზი N1)



პროექტის კონსტრუქციის საჩვენებლად OpenSCAD- ის პლატფორმაზე დავაპროგრამეთ და ავაწყეთ 3D მოდელი. თავშესაფრის მოდელი იმეორებს რეალური კონსტრუქციის არქიტექტურას. კორპუსი არის ხის, 50x50x50 სმ. აქვს ორქანობიანი სახურავი 20 სმ წინა გამოშვებით, იმისთვის რომ კამერა დაიცვას წვიმისგან. შესასვლელი არის თაღოვანი, 20 სმ სიგანეში და 30 სმ სიღრმეში. შესასვლელი არის თავშესაფრის კუთხეში, რომ საჭიროების შემთხვევაში მიუსაფარმა ცხოველმა კედლის მხარეს შეძლოს ქარისგან თავის შეფარება. კონსტრუქციას აქვს 10 სმ სიმაღლის ფეხები.

[3D ნახაზი \(Git-Hub\)](#)

2. სტრუქტურული და ფუნქციონალური ბლოკ-სქემა (ნახაზი N2)



[ბლოკ სქემის ლინკი](#)

ქვეყანი თავშესაფრის სტრუქტურული და ფუნქციონალური ბლოკ-სქემა იყოფა სამ ნაწილად:

სენსორების მართვის მოდული - 1Hz სიხშირით აგროვებს მონაცემებს მანძილის და ტემპერატურის სენსორებიდან და UART პროტოკოლით აწვდის BluePill-ს.

მონაცემების დამუშავება და კავშირი - ESP32 სენსორულ მონაცემებს JSON ფორმატში აგზავნის Digital Ocean -ის GO სერვერზე, ვიდეონაკადს RTSP პროტოკოლით სტრიმავს MediaMTX-ზე.

გათბობის მართვის სისტემა - ESP32, 2N2222A ტრანზისტორის გავლით მართავს რელეს, რომელიც რთავს 12V/30W გამათბობელ სტრინგებს.

4.2 დიზაინის დეტალური აღწერა

LilyGo T-SIMCAM (ESP32-S3)

LilyGo T-SIMCAM არის ESP32-S3 ჩიპზე დაფუძნებული პლატა, რომელსაც ჩაშენებული აქვს OV5640 მოდელის კამერა. მუშაობს ESP32-S3 dual-core Xtensa LX7 პროცესორზე [7], 240MHz სიხშირეზე, აქვს 8 MB Flash და 8 MB PSRAM. ეს პლატა არის სისტემის მთავარი მოდული. პლატა დამოუკიდებლად უკავშირდება Wi-Fi-ს, იღებს სტრიმს, იღებს სენსორულ მონაცემებს BluePill-ისგან და მართავს რელეს.

FreeRTOS - ზე რამდენიმე ტასკი მუშაობს. bp_link_task კითხულობს BluePill -ის UART სტრიქონს. thermostat_task მართავს გათბობას. sensor_task JSON- ს გზავნის სერვერზე და rtsp_publisher_task ვიდეოს MediaMTX სერვერზე სტრიმავს.

STM32F103 BluePill

STM32F103C8T6, ანუ იგივე BluePill არის ARM Cortex-M3 არქიტექტურაზე დაფუძნებული მიკრომარშრუტიზატორი. [6] მისი ამოცანაა წაიკითხოს მონაცემები VL53L1X და SHT40 - სენსორებისგან, დაამუშაოს და გაუგზავნოს ESP32-S3-ს. მუშაობს 72 MHz სიხშირეზე. BluePill -ს გააჩნია UART და I2C ინტერფეისებს. პროგრამული უზრუნველყოფა დაიწერა C ენაზე, STM32CubeIDE გარემოში, HAL ბიბლიოთეკის გამოყენებით. BluePill ყოველ ერთ წამში აგროვებს ახალ მონაცემებს და UART2-ით გადასცემს LILYGo-ს.

შევარჩიეთ, რადგან LILYGo პლატაზე არ იყო საკმარისი UART და I2C პინები.

VL53L1X

VL53L1X არის STMicroelectronics-ის ToF მანძილის სენსორი, რომელიც BluePill-ს I2C პროტოკოლით უკავშირდება. სენსორი ინფრაწითელ ლაზერულ სხივს აგზავნის და არეკლილი სინათლის დაბრუნების დროს ზომავს. პროექტში სენსორი Long Mode- ში მუშაობს. [3] მოთავსებულია თავშესაფრის შიგნით, ჭერში და 500მმ ზღვარს იყენებს, იმის დასადგენად არის თუ არა თავშესაფარში ცხოველი. თუ უახლოეს ობიექტამდე მანძილი 500მმ-ზე ნაკლებია, სისტემა ასკვნის რომ თავშესაფარი დაკავებულია.

ToF სენსორი შევარჩიეთ ულტრაბერითის ნაცვლად, რადგან მუშაობს ± 1 მმ სიზუსტით, გაცილებით პატარაა და უფრო მარტივად თავსდება სახლის შიგნით, ჩუმია და ცხოველს არ აღიზიანებს.

SHT40

SHT40 არის Sensirion ტემპერატურა-ტენიანობის საზომი სენსორი. გამოიყენება UART bridge მოდულის სახით. ჩვენს შემთხვევაში ვიყენებთ მხოლოდ ტემპერატურის საზომად. მოდული BluePill-ის PA10 პინზე (USART1 RX) აგზავნის მონაცემებს შემდეგი ფორმატით: R:041.6RH 024.7C

SHT40 შევარჩიეთ DHT22-ის ნაცვლად, რადგან აქვს უფრო მაღალი სიზუსტე - $\pm 0.2^{\circ}\text{C}$. [4] UART bridge მოდული I2C-ზე უკეთესია, რადგან I2C BluePill-ზე ToF სენსორის მიერაა დაკავებული.

რელე

12V გამათბობელი ელემენტები იმართება ოპტოიზოლირებული რელეს საშუალებით. ESP32-ს GPIO21 სიგნალი მიდის ტრანზისტორის ბაზაზე, ტრანზისტორი ოპტოიზოლატორის ლედს კვებავს, ოპტოიზოლატორი რელეს სარქველს მართავს.

SSR რელეს ნაცვლად შევარჩიეთ ოპტოიზოლირებული რელე, იმიტომ რომ უფრო იაფია, მატრევად ინტეგრირდება და ამ სიმძლავრისთვის საკმარისია.

Go Backend + ვებსაიტი

Go ენაზე დაწერილი სერვერი Digital Ocean-ის VM-ზე გაეშვა. TCP:5678 ESP32-ისგან იღებს მონაცემებს - JSON ტიპის. HTTP:8080 კი აწვდის ვებსაიტს. ვებსაიტი ჩაშენებულია Go binary-ში //go:embed static დირექტივით. ვებსაიტზე აისახება ვიდეოსტრიმი WebRTC ფორმატში და გამოჩნდება სენსორული მონაცემები და დონაციის პანელი.

Go შეირჩა მარტივი მოდელის გამო, მისი goroutine-ები საშუალებას გვაძლევს ათასობით კავშირი ერთდროულად დავამუშაოთ. ვებსაიტი პირდაპირ სერვერის პროგრამაშია ჩაშენებული, ამიტომ სერვერზე მხოლოდ ერთი ფაილის გადატანა სჭირდება და დამატებითი კონფიგურაცია აღარაა საჭირო.

4.3 ანალიზის შედეგები

საბოლოო დიზაინი დაფუძნებულია რამდენიმე საინჟინრო გადაწყვეტილებაზე. ESP32-S3-ის არასაკმარისი რაოდენობის GPIO პინებმა განაპირობა მეორე მიკროკონტროლერის დამატება. STM32F103 დაემატა სენსორების მოდულად. ენერგობიუჯეტის ანალიზმა დაადასტურა რომ შერჩეული კვების წყაროები საკმარისია. ელექტრონიკა სულ 1W-ს მოიხმარს, გამათბობელი სტრინგები კი 30W-ს. 30W სტრინგები და ცხოველის სხეულის სითბო ერთად საკმარისია სახლის შიგნიდან სითბოს შესანარჩუნებლად.

UART კავშირის ანალიზით დავადასტურეთ, რომ BluePill-სა და ESP32-ს შორის მონაცემების გადაცემა ხდება 44მწ-ში, რაც ერთ წამიანი პერიოდისთვის საკმარისია. თერმოსტატი გათბობის მდგომარეობას(ჩართვა-გამორთვას) 1.5 წამში ცვლის, რაც ტემპერატურის ცვლილების სიჩქარის გათვალისწინებით მისაღებია.

4.4 ღირებულების ანალიზი

პროექტისთვის გამოყოფილი თანხა დაიხარჯა სახლის კონსტრუქციისთვის შესაძენ მასალაზე და სისტემის ასაწყობად შეძენილ ელექტრონიკაზე. ჯამურმა დანახარჯმა შეადგინა 378.09 ლარი. ელექტრონიკაში ყველაზე ძვირი კომპონენტია LilyGo T-SIMCAM პლატა. სენსორები, რელე და BluePill შედარებით იაფია. კონსტრუქციული მასალებიდან ძირითადი ხარჯია ხის ფანერი. საბოლოო ჯამში მიზანს მინიმალური დანახარჯებით მივაღწიეთ.

ცხრილი N7

მოწყობილობა / დეტალი	რაოდ.	ფასი / ერთეული (ლარი)	ჯამი (ლარი)
LilyGo T-SIMCAM (ESP32-S3)	1	84.49 ლარი	84.49 ლარი
STM32F103C8T6 BluePill	1	18 ლარი	18 ლარი
VL53L1X ToF სენსორი	1	39 ლარი	39 ლარი
SHT40 სენსორი	1	18 ლარი	18 ლარი
რელე	1	12 ლარი	12 ლარი
12V გამათბობელი ელემენტები	15	5 ლარი	75 ლარი
12V/5A DC ადაპტერი	1	25 ლარი	25 ლარი
5V/2A USB ადაპტერი	1	12 ლარი	12 ლარი
ხის ფანერი 50x50 სმ	4	15 ლარი	60 ლარი
ტოლის ნაჭერი (სახურავისთვის)	2	6 ლარი	12 ლარი
2N2222A ტრანზისტორი	1	0.50 ლარი	0.50 ლარი
მავთულები(Jumper wires)	20	0.30 ლარი	6 ლარი
სამონტაჟო ყუთი	1	16 ლარი	16 ლარი
რეზისტორი 1k Ω (R_B)	1	0.10 ლარი	0.10 ლარი
ჯამი			378.09 ლარი

[ბიუჯეტის ფაილის ლინკი\(Git-Hub\)](#)

4.5 მასალის, გეომეტრიისა და კომპონენტების შერჩევა

ხის კონსტრუქცია

თავშესაფრის ასაგებად გამოვიყენეთ ხის ფანერა და სახურავის გადასაფარად გამოვიყენეთ ტოლი. ხე შევარჩიეთ , რადგან არის კარგი თბოიზოლატორი. ტოლი გამძლეა გარე პირობებში, წვიმის, თოვლის, ყინვის დროს. თავშესაფრის ზომა არის 50x50x50. გათვლილია საშუალო ზომის ძაღლზე ან კატაზე. საჭიროების შემთხვევაში კორპუსის ზომები მარტივად ადაპტირებადია უფრო დიდი ჯიშის ცხოველებისთვის.

ორქანობიანი სახურავი

სახურავის დახრილი ფორმა უზრუნველყოფს ნალექის(წვიმა, თოვლი) მარტივ ჩამოდინებას და ხელს უშლის მის დაგროვებას ზედაპირზე. ფასადზე არსებული 20 სმ-იანი გამოშვება კი წინა კედელზე დამონტაჟებულ კამერას იცავს ნესტისგან.

ფეხები

10 სმ სიგრძის ფეხები კონსტრუქციას აშორებს მიწის ზედაპირს, შესაბამისად ტენიანობა ვერ აღწევს კონსტრუქციამდე და მუდმივად ინარჩუნებს სიმშრალეს.

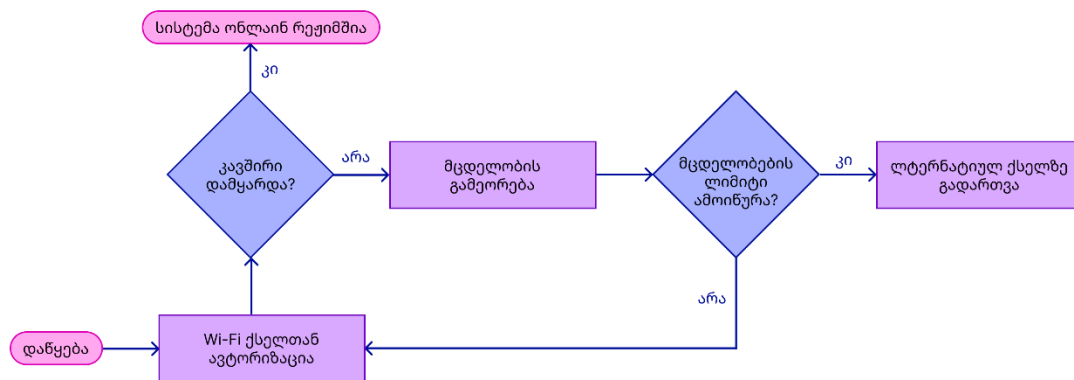
თაღოვანი შესასვლელი

შესასვლელი განთავსებულია თავშესაფრის კუთხესთან ახლოს, რათა გვერდითა კედელმა მიუსაფარი ცხოველი ქარისგან დაიცვას. შესასვლელის თაღოვანი ფორმა და მახვილი კუთხეების არარსებობა იცავს ცხოველს დაშავებისგან და კარში ბეწვის გამოდებისგან.

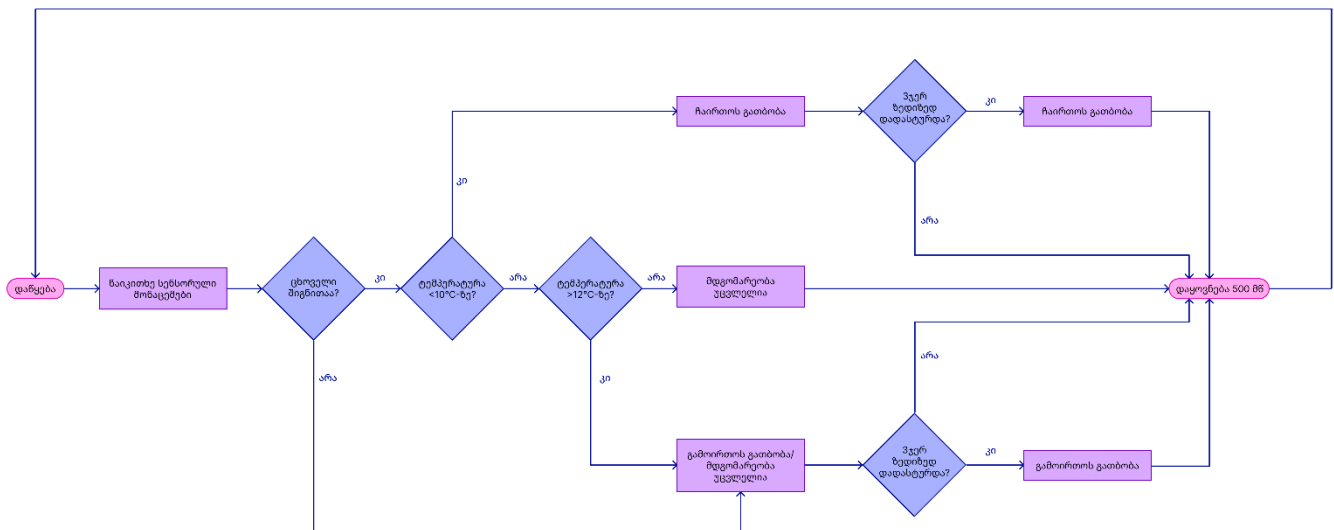
4.6 ბლოკ-სქემები, ელექტრო-სქემები, ფსევდოკოდი და მონტაჟის სქემები

პროგრამის ბლოკ-სქემები (ნახაზი N3)

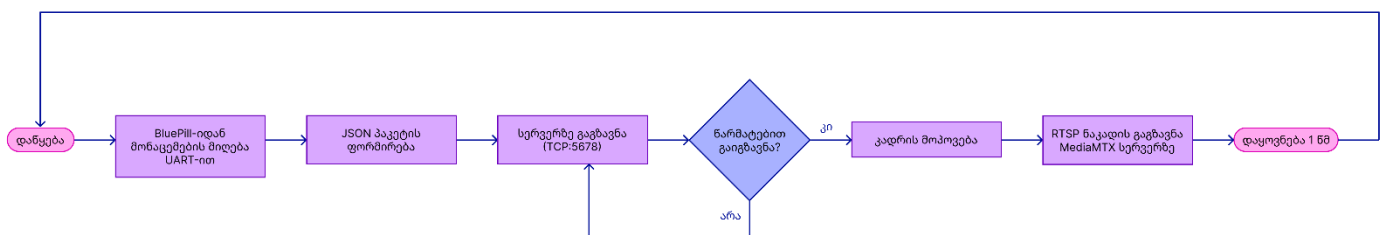
1. Wi-Fi კავშირის დამყარების ბლოკ-სქემა



2.თერმოსტატის ალგორითმის ბლოკ-სქემა (ნახაზი N4)

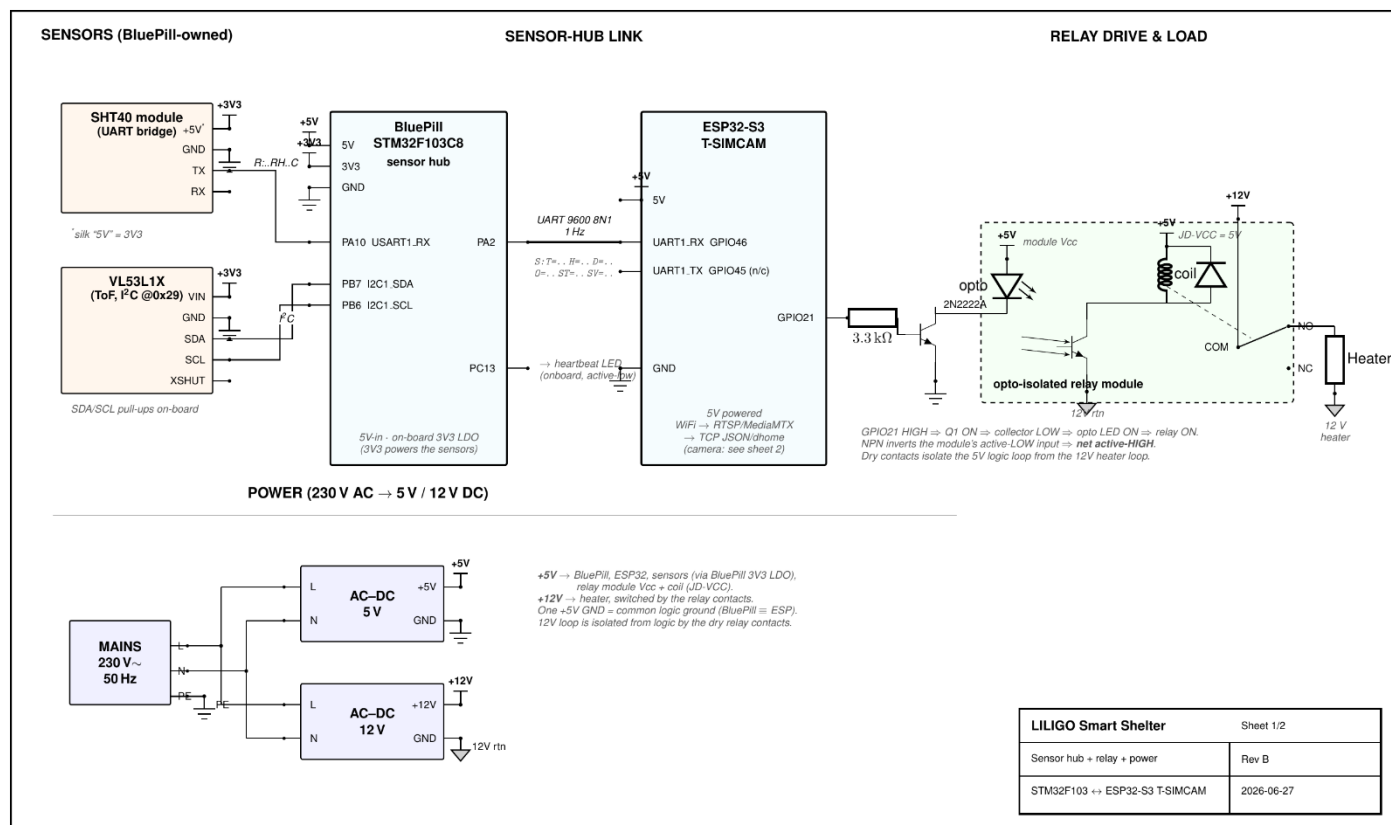


3. მონაცემების გადაცემის ბლოკ-სქემა(ნახაზი N5)



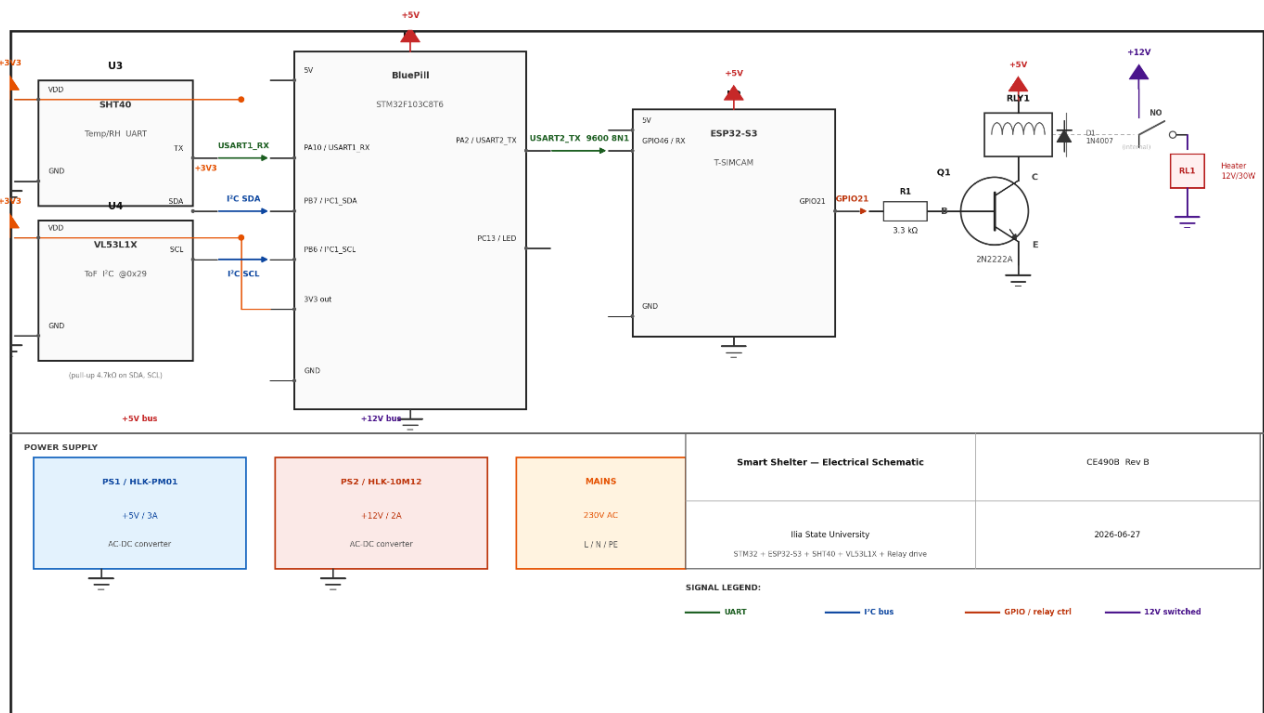
[ბლოკ-სქემების ლინკი\(Git-Hub\)](#)

ელექტრო-სქემა (ნახაზი N6)



ელექტრო-სქემის ლინკი (Git-Hub)

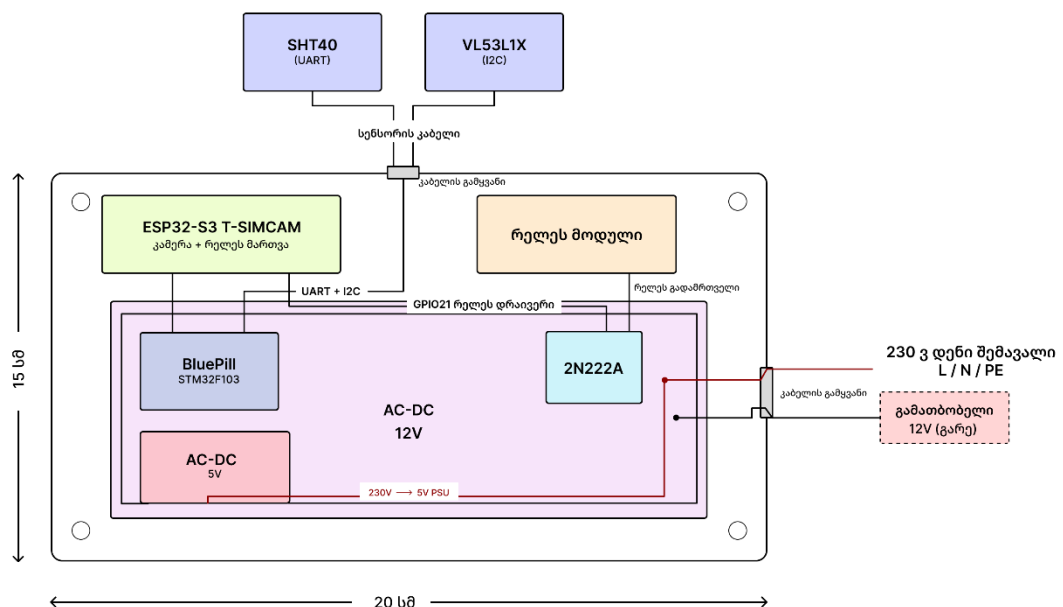
ელექტრული წრედი(ნახაზი N7)



ელექტრული წრედის ლინკი

მონტაჟის სქემა (ნახაზი N8)

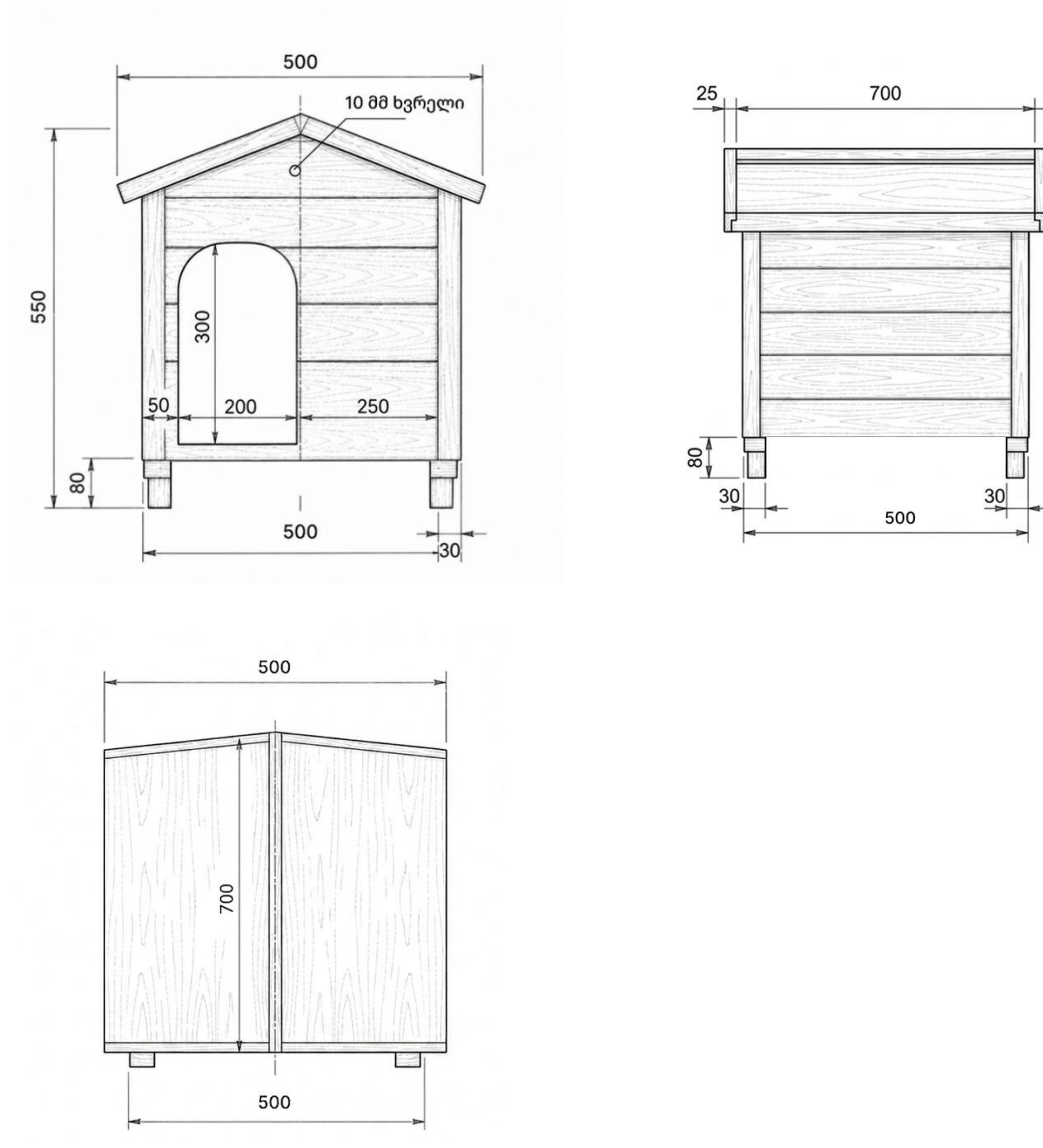
კომპონენტების განლაგება (ზედა ხედი)



მონტაჟის სქემის ლინკი

4.7 საწარმოო ნახაზები

ნახაზი N9



მასალები - ხის ფიგარი (კედლები, იატაკი, ფეხები) და ტოლი (სახურავი).

ზომები - წინა კედელი: 550 მმ - სიგრძე, 500 - სიგანე

გვერდითა კედლები: 420 მმ სიგრძე, 500 მმ სიგანე

სახურავი: 700 მმ სიგრძე, 360 მმ სიგანე

[საწარმოო ნახაზები ლინკი](#)

4.8 უსაფრთხოების მოსაზრებები

ელექტრული უსაფრთხოება

12V და 5V წრედები ერთმანეთისგან გამოყოფილია. გათბობის სტრინგებს მართავს ოპტიზოლირებული რელე, რომელიც 5V საკონტროლო წრედს 12V წრედისგან გალვანურად განაცალკევებს. ESP32-ის GPIO21 პინი დაცულია 12V ძაბვის პირდაპირი ზემოქმედებისგან, მართვის სიგნალი გადაიცემა 2N2222A ტრანზისტორის გავლით.

თერმული უსაფრთხოება

თერმოსტატი ავტომატურად თიშავს გათბობას ტემპერატურის 12°C-ზე ზემოთ ასვლის ან თავშესაფრის დაცარიელების შემთხვევაში. გამათბობელი ელემენტები მოთავსებულია ხის იატაკის ქვეშ და ცხოველთან არ აქვს პირდაპირი კონტაქტი. 30W სიმძლავრე 50x50x50 ზომის კონსტრუქციისთვის სრულად უსაფრთხოა და არ წარმოადგენს გადახურების საფრთხეს.

მექანიკური უსაფრთხოება

ელექტრონიკის გარემო პირობებისგან დაცვას თავად ხის კონსტრუქცია უზრუნველყოფს.

მონაცემთა უსაფრთხოება

სერვერთან კავშირი მყარდება TCP პროტოკოლის გამოყენებით. ვებგვერდზე წვდომა ხდება უშუალოდ IP მისამართით.

4.9 მოვლისა და შეკეთების მოსაზრებები

პროგრამული განახლება

ESP32-ის პროგრამული განახლება ხდება USB კაბელის მეშვეობით, STM32CubeIDE და ESP-IDF გარემოში. Go სერვერის განახლებისთვის კი საჭიროა ახალი ბინარული ფაილის ატვირთვა DigitalOcean- ის სერვერზე.

სენსორების შემოწმება

VL53L1X და SHT40 სენსორების მუშაობის შემოწმება შესაძლებელია ვებსაიტზე მათ მიერ ასახული მონაცემების მონიტორინგით. იმ შემთხვევაში თუ მონაცემები არ განახლდება, უნდა შემოწმდეს BluePill და ESP32 პლატებს შორის UART კავშირი.

გათბობის ელემენტები

გათბობის ელემენტების სტრუქტურა საშუალებას იძლევა დაზიანებული ელემენტი ცალკე შეიცვალოს, მთლიანი სისტემის დაშლის გარეშე. ელემენტის შეცვლა მოითხოვს 12V კვების წყაროს გამორთვას.

კონსტრუქცია

ხის კონსტრუქცია წელიწადში ერთხელ უნდა შემოწმდეს ნესტზე და ფიზიკურ დაზიანებებზე. საურავის დამცავი ტოლი კი საჭიროების შემთხვევაში განახლდეს.

სათადარიგო ნაწილები

სისტემის უწყვეტი მუშაობისთვის რეკომენდებულია რამდენიმე დამატებითი გათბობის ელემენტს სათადარიგოდ შენახვა.

[GitHub-ის ლინკი](#)

თავი 5. დიზაინის ვერიფიკაციის გეგმა (ტესტირება)

5.1 ტესტების აღწერა

ტესტი N1: UART კავშირი BluePill-სა და ESP32-S3-ს შორის.

მიზანი: დავადასტუროთ, რომ STM32F103 სენსორულ მონაცემებს სწორად გადასცემს ESP32-ს 9600 baud სიჩქარით.

პროცედურა: ESP32 BluePill- ს დაუკავშირდა Serial Monitor Arduino IDE -ს საშუალებით. ESP32-ის მიერ მიღებული სტრიქონი შევადარეთ გაგზავნილ ფორმატს.

გამოყენებული აღჭურვილობა: USB-Serial adapter, Arduino IDE Serial Monitor

მისაღები კრიტერიუმი: მონაცემები გამოტოვების გარეშე მიდის 1 წამის ინტერვალით.

შედეგი: ტესტი წარმატებულია. მონაცემები სწორი ფორმატით მიდიოდა და გამოტოვებაც არ დაფიქსირებულა.

ტესტი N2: VL53L1X მანძილის სენსორი

მიზანი: განვსაზღვროთ რომ სენსორი 500მმ-ზე ნაკლებ მანძილს სწორად განსაზღვრავს.

პროცედურა: თითო(ან ნებისმიერი ნივთი) მოვათავსოთ სენსორის წინ სხვადასხვა მანძილზე, მაგ. 200 მმ, 400 მმ, 600 მმ და მონიტორში წავიკითხოთ მნიშვნელობები.

გამოყენებული აღჭურვილობა: საზომი ლენტი, Arduino IDE.

მისაღები კრიტერიუმი: 500 მმ-ზე ნაკლებ მანძილზე იწერება რომ თავშესაფარი დაკავებულია.

შედეგი: ტესტი წარმატებულია.

ტესტი N3: SHT40 ტემპერატურის სენსორი

მიზანი: ტემპერატურის გაზომვის სიზუსტის შემოწმება

პროცედურა: SHT40-ის მაჩვენებელი შევადარეთ თერმომეტრს ოთახის ტემპერატურაზე

გამოყენებული აღჭურვილობა: თერმომეტრი, Serial Monitor

მისაღები კრიტერიუმი: განსხვავება უნდა მოგვცეს $\leq 1^{\circ}\text{C}$

შედეგი: ტესტი წარმატებულია, სხვაობა იყო 0.3°C -ის ფარგლებში.

5.2 სპეციფიკაციების შემოწმების სია

ცხრილში მოაქციეთ 1-ელი თავის თითოეული სპეციფიკაცია მისი ვერიფიკაციის შედეგთან ერთად. ქვემოთ მოცემული ცხრილი საწყისი წერტილია.

ცხრილი N8

#	სპეციფიკაცია	სამიზნე	გაზომილი	ჩაბ. / ჩავარდ.
1	ტემპერატურა	$<10^{\circ}\text{C}$ ჩართვა, $>12^{\circ}\text{C}$ გამორთვა	კოდით დადასტურებულია	ჩაბ.
2	სენსორის სიზუსტე	$\pm 1^{\circ}\text{C}$	$\pm 0.3^{\circ}\text{C}$ (ტესტი N3)	ჩაბ.
3	რეაგირების დრო	≤ 1.5 წმ	1.5 წმ ($3 \times 500\text{მს}$)	ჩაბ.
4	ვიდეოსტრიმის დაყოვნება	≤ 3 წმ	2-3 წმ	ჩაბ.
5	ცხოველის დაფიქსირება	$\leq 500\text{მმ}$ მანძილი	500მმ ზღვარი კოდში	ჩაბ.
6	სისტემის სიმძლავრე	$\leq 31\text{W}$	30.17W (ენერგობიუჯეტი)	ჩაბ.
7	სერვერზე მონაცემების გაგზავნა	≤ 1 წმ	1.2 წმ (ქსელის გადატვირთვისას)	ჩავარდ.

[სპეციფიკაციების ფაილის ლინკი \(Git-Hub\)](#)

თავი 6. დასკვნები და რეკომენდაციები

6.1 პროექტის გრაფიკი და ამოცანების განაწილება

ამოცანა	სტუდენტი	სტატუსი	მარტი				აპრილი				მაისი				ივნისი			
			კვ.1	კვ.2	კვ.3	კვ.4	კვ.1	კვ.2	კვ.3	კვ.4	კვ.1	კვ.2	კვ.3	კვ.4	კვ.1	კვ.2	კვ.3	კვ.4
1. პროგრამული უზრუნველყოფა																		
ESP32-S3 -ის პროგრამული უზრუნველყოფა	თამარ ჯაში	დაგეგმილი																
		რეალური																
BluePill-ის პროგრამული უზრუნველყოფა	თამარ ჯაში	დაგეგმილი																
		რეალური																
Wi-Fi კომუნიკაციის გამართვა	მარი დავლაძე	დაგეგმილი																
		რეალური																
ვიდეო სტრიმის განხორციელება	გრიშა ტონერიანი	დაგეგმილი																
		რეალური																
2. სერვერი და ვებინტერფეისი																		
Go backend-ის დაწერა	გრიშა ტონერიანი, მარი დავლაძე	დაგეგმილი																
		რეალური																
Frontend-ის დაწერა/ინტერფეისის დიზაინი	თამარ ჯაში	დაგეგმილი																
		რეალური																
სერვერის კონფიგურაცია	მარი დავლაძე	დაგეგმილი																
		რეალური																
3. აპარატურა და მონტაჟი																		
სქემის დიზაინი	თამარ ჯაში	დაგეგმილი																
		რეალური																
სახლის კონსტრუქციის აწყობა	ყველა	დაგეგმილი																
		რეალური																
ელექტრონიკის მონტაჟი	ყველა	დაგეგმილი																
		რეალური																
4. ტესტირება																		
სენსორების ტესტი	გრიშა ტონერიანი	დაგეგმილი																
		რეალური																
თერმოსტატის ტესტი	თამარ ჯაში	დაგეგმილი																
		რეალური																
5. დოკუმენტაცია																		
პროექტის საბოლოო ანგარიში	ყველა	დაგეგმილი																
		რეალური																
პრეზენტაციის მომზადება	ყველა	დაგეგმილი																
		რეალური																

ცხრილი N9

[განტის დიაგრამის ფაილის ლინკი \(Git-Hub\)](#)

6.2 ინდივიდუალური წვლილი

ცხრილი N10

გუნდის წევრი	როლ(ებ)ი	ძირითადი წვლილი
თამარ ჯაში	ტექნიკური ლიდერი (Hardware/Frontend), სქემის დიზაინერი, პროგრამული უზრუნველყოფის დეველოპერი, პროექტის მენეჯერი, თანარედაქტორი.	შექმნა მიკროკონტროლერების პროგრამული უზრუნველყოფა, სქემების დიზაინი და მომხმარებლის ინტერფეისი (Frontend). მართავდა პროექტის მიმდინარეობასა და გრაფიკს. გატესტა თერმოსტატი. გუნდთან ერთად დაწერა საბოლოო ანგარიში და მოამზადა პრეზენტაცია.
გრიშა ტონერიანი	ტექნიკური ლიდერი (Backend/Streaming), Backend დეველოპერი, თანარედაქტორი.	მარისტან ერთად დაწერა პროექტის სერვერული (Go Backend) ნაწილი, გამართა ვიდეო სტრიმი და გატესტა სენსორები. გუნდთან ერთად დაწერა საბოლოო ანგარიში.
მარი დავლაძე	ქსელური უზრუნველყოფის სპეციალისტი, სისტემური ადმინისტრატორი, Backend დეველოპერი, ბიუჯეტის მენეჯერი, თანარედაქტორი.	გამართა ქსელი (Wi-Fi), მოამზადა სერვერი და გრიშასთან ერთად დაწერა სერვერული ნაწილის კოდი (Go backend). პასუხისმგებელი იყო კომპონენტების შესყიდვასა და ბიუჯეტის მართვაზე. გუნდთან ერთად დაწერა საბოლოო ანგარიში.

[ინდივიდუალური წვლილი \(ცხრილი\) – Git-Hub](#)

გამოყენებული წყაროები

- [1] K&H Pet Products, "Outdoor Heated Cat House," [Online]. Available: <https://khpets.com>, Accessed: June 2026.
- [2] PetKit, "Cozy Smart Pet House," [Online]. Available: <https://www.petkit.com>, Accessed: June 2026.
- [3] STMicroelectronics, "VL53L1X Datasheet," DocID031281 Rev 3, November 2018, [Online]. Available: [VL53L1X Datasheet](#)
- [4] Sensirion, "SHT40 Datasheet," Version 4, 2021. [Online]. Available: [SHT40 Datasheet](#)
- [5] H. Schulzrinne, A. Rao, R. Lanphier, "RFC 2326 - Real Time Streaming Protocol," IETF, April 1998. [Online]. Available: [RFC 2326 Editor](#)
- [6] STMicroelectronics, "STM32F103C8 Datasheet," 2015. [Online]. Available: [STM32F103 Datasheet](#)
- [7] Espressif, "ESP32-S3 Datasheet," V1.4, 2023. [Online]. Available: [ESP32-S3 Datasheet](#)
- [8] LILYGO, "T-SIMCAM Product Page," 2022. [Online]. Available: <https://lilygo.cc/en-us/products/t-simcam>
- [9] OmniVision Technologies, "OV5640 Color CMOS QVGA (5 Megapixel) Image Sensor Datasheet," 2011. [Online]. Available: [OV5640](#)
- [10] Ningbo Songle Relay, "SRD-05VDC-SL-C Relay Datasheet," [Online]. Available: [SRD-05VDC-SL-C](#)

დანართები

- დანართი A – კომპონენტების სპეციფიკაციები და datasheet-ები.

ცხრილი N11

კომპონენტი	მოდელი	სპეციფიკაციები	წყარო
მიკროკონტროლერი	STM32F103C8T6	72MHz, 64KB Flash, 20KB SRAM	datasheet
მიკროკონტროლერი/კამერა	LilyGo T-SIMCAM(ESP32-S3)	240MHz, 8MB PSRAM, Wi-Fi	პროდუქტის გვერდი
მანძილის სენსორი	VL53L1X	ToF, 4m მაქს., I2C	datasheet
ტემპერატურის სენსორი	SHT40	±0.2°C, I2C	datasheet
კამერა	OV5640	5MP, MIPI	datasheet
ტრანზისტორი	2N2222A	NPN, 600mA, 40V	datasheet
რელე	SRD-05VDC-SL-C	5V, 10A/250VAC	datasheet

- დანართი B – დეტალური ანალიზის შედეგები და გამოთვლები.

ბაზის რეზისტორის გამოთვლა (2N2222A)

GPIO ძაბვა: $V_{GPIO} = 3.3V$

ბაზა-ემიტერის ძაბვა: $V_{BE} = 0.7V$

რელეს კოჭის დენი: $I_C = 5mA$

გამდიერების კოეფიციენტი: $h_{FE} = 100$

საჭირო ბაზის დენი (10x ზღვარი): $I_B = (I_C / h_{FE}) \times 10 = (5mA / 100) \times 10 = 0.5 mA$

$R_B = (3.3 - 0.7) / 0.0005 = 5200 \Omega$

გამოყენებულია 3.3kΩ

$I_B = 2.6 / 3300 = 0.79 mA$

- დანართი C – სრული საწარმოო ნახაზები (ასევე წარდგენილი ცალკე და GitHub-ზე).

[საწარმოო ნახაზები\(Git-Hub\)](#)

- დანართი D – მომხმარებლის სახელმძღვანელო / საექსპლუატაციო დოკუმენტაცია.
მომხმარებლის სახელმძღვანელო

[მომხმარებლის სახელმძღვანელოს ლინკი - Git-Hub](#)

- დანართი E – საწყისი კოდის შერჩეული ფრაგმენტები (სრული კოდი GitHub-ზე).

თერმოსტატის ლოგიკა

```
static void thermostat_task(void *arg) {
    int on_hits = 0, off_hits = 0;
    while (1) {
        float temp = 0, rh = 0;
        if (bp_link_get_temp_humidity(&temp, &rh) == ESP_OK) {
            bool occupied = false;
            bool occ_known = (bp_link_get_occupied(&occupied) ==
ESP_OK);

            bool vacant = occ_known && !occupied;
            bool cold = temp < (float)CONFIG_RELAY_TEMP_MIN_C;
            bool hot = temp > (float)CONFIG_RELAY_TEMP_MAX_C;

            bool want_on = cold && !vacant;
            bool want_off = hot || vacant;

            on_hits = want_on ? on_hits + 1 : 0;
            off_hits = want_off ? off_hits + 1 : 0;

            if (!relay_get() && on_hits >= DEBOUNCE_HITS) {
                relay_set(true); on_hits = 0;
            } else if (relay_get() && off_hits >= DEBOUNCE_HITS) {
                relay_set(false); off_hits = 0;
            }
        } else {
            on_hits = 0; off_hits = 0;
        }
    }
}
```



```

        vTaskDelay(pdMS_TO_TICKS(POLL_PERIOD_MS));
    }
}

```

BluePill-ის ESP32-თან UART კავშირი

```

static void bp_link_uart_task(void *arg) {
    char line[BP_LINK_LINE_MAX];
    int len = 0;
    uint8_t chunk[64];
    while (1) {
        int n = uart_read_bytes(BP_LINK_UART, chunk, sizeof(chunk),
                                pdMS_TO_TICKS(500));

        if (n <= 0) continue;
        for (int i = 0; i < n; i++) {
            uint8_t b = chunk[i];
            if (b == '\r') continue;
            if (b == '\n' || len >= BP_LINK_LINE_MAX - 1) {
                line[len] = '\0';
                float t, rh;
                unsigned int mm, occ, st, sv;
                if (sscanf(line,
                           "S:T=%f H=%f D=%u O=%u ST=%u SV=%u",
                           &t, &rh, &mm, &occ, &st, &sv) == 6)
                    publish_reading(t, rh, (uint16_t)mm,
                                    occ != 0, st != 0, sv != 0);

                len = 0;
            } else {
                line[len++] = (char)b;
            }
        }
    }
}

```

სენსორის მონაცემების გაგზავნა სერვერზე

```
static void sensor_task(void *arg) {
    while (true) {
        int sock = dhome_connect();
        if (sock < 0) { vTaskDelay(pdMS_TO_TICKS(backoff * 1000));
continue; }

        while (true) {
            float temp = 0, rh = 0;
            uint16_t dist = 0;
            bool occupied = false;
            bool t_ok = (bp_link_get_temp_humidity(&temp, &rh) ==
ESP_OK);

            bool d_ok = (bp_link_get_distance_mm(&dist) == ESP_OK);
            bp_link_get_occupied(&occupied);

            char line[256];
            snprintf(line, sizeof(line),
                    "{ \"temperature\":%.2f, \"humidity\":%.2f, \"
distance_mm\":%u, \"occupied\":%s, \"status\":%s} \n",
                    temp, rh, (unsigned)dist,
                    occupied ? "true" : "false",
                    (t_ok && d_ok) ? "true" : "false");

            if (send(sock, line, strlen(line), 0) <= 0) break;
            vTaskDelay(pdMS_TO_TICKS(CONFIG_SENSOR_PERIOD_MS));
        }
        close(sock);
    }
}
```

RTSP ვიდეოსტრიმი

```
while (true) {
    camera_fb_t *fb = esp_camera_fb_get();
    if (!fb) { vTaskDelay(pdMS_TO_TICKS(10)); continue; }
    if (fb->format == PIXFORMAT_JPEG) {
        uint32_t ts = (uint32_t)((esp_timer_get_time() - t0) * 9LL /
100LL);
        rtp_mjpeg_send_frame(tcp_interleaved_sink, &sink_ctx,
                                fb->buf, fb->len, &seq, ts, ssrc);
    }
    esp_camera_fb_return(fb);
}
```

სრული კოდები Git- Hub- ზე

[ESP32 კოდი](#)

[Go backend კოდი](#)

- დანართი F – სტუდენტის განცხადება და წვლილის ჩანაწერები (იხ. სილაბუსის დანართი #4).

[თამარ ჯაში](#)

[გრიშა ტონერიანი](#)

[მარი დავლაძე](#)